

UNIT-III

①

Deep feed forward Networks

* Deep feed fwd n/w's are also called as feed forward neural networks (or) multilayer perceptrons (MLP's)

→ The goal of a feed forward n/w is to approximate the

fn: f^*

i.e for a classifier, $y = f^*(x)$ maps i/p x to category y .

→ A feed forward n/w defines a mapping $y = f(x; \theta)$
It learns value of θ that ~~are~~ result in best function approximation.

→ These models are called feed forward because info. flows in forward direction.
(i.e from x , through intermediate computations used to define f , & finally to o/p y)

→ These are called n/w's because they are represented by composing together many different functions.

for ex: If we have 3 fns $f(1)$, $f(2)$, $f(3)$ connected -
we write it as $f(x) = f(3)(f(2)(f(1)(x)))$

here $f(1)$ is called first layer
 $f(2)$ " " second layer & so-on

→ The overall length of the chain, gives depth of the model.
So the name deep learning.

→ final layer is called output layer.

→ The training examples specify directly what the output layer must do at each point x . It must produce a value close to y .

→ The behaviour of the other layers is not directly specified by training data.

→ So these are called hidden layers

* → One way to understand feed forward n/w is to begin with linear models.

* → To overcome the limitations of linear models, we can apply non linear transformations.

→ Feed forward n/w's have introduced the concept of hidden layers & this requires us to choose Activation fns that will be used to compute the hidden layer values.

* Example : Learning XOR

So, let's begin with an example of a fully connected feed forward n/w, i.e. learning the XOR function.

XOR fn:	x_1	x_2	y
	0	0	0
	0	1	1
	1	0	1
	1	1	0

It provides the target fn: $y = f^*(x)$ that we want to learn.

→ our model provides a fn: $y = f(x; \theta)$

∴ learning alg. will adapt the parameters θ to make f as similar as f^*

→ As this is a regression problem we use Mean Squared error loss fn: (MSE).

* evaluate this on our whole training set

MSE loss fn: is

$$J(\theta) = \frac{1}{4} \sum_{x \in X} (f^*(x) - f(x; \theta))^2 \quad \text{--- (1)}$$

[0, 0]
[0, 1]
[1, 0]
[1, 1]

→ function of our model is $f(x; \theta)$

with θ consisting of w & b
↳ wt ↳ bias

* our model is defined as

$$f(x; w, b) = x^T w + b \quad \text{--- (2)}$$

below is the fig. with explanation after applying transformation.

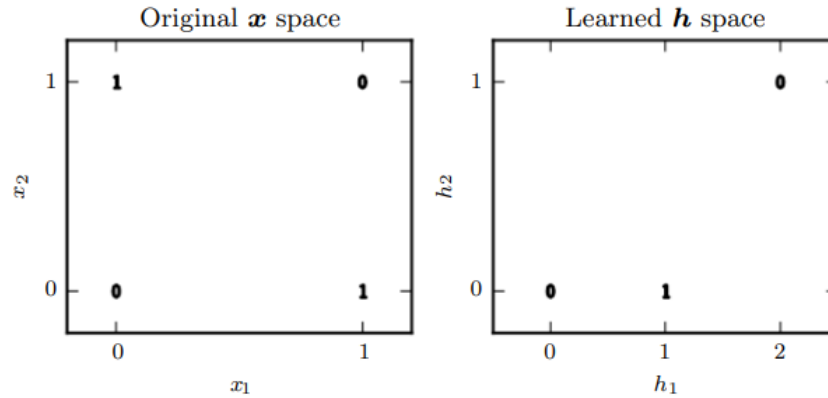


Figure 6.1: Solving the XOR problem by learning a representation. The bold numbers printed on the plot indicate the value that the learned function must output at each point. *(Left)* A linear model applied directly to the original input cannot implement the XOR function. When $x_1 = 0$, the model's output must increase as x_2 increases. When $x_1 = 1$, the model's output must decrease as x_2 increases. A linear model must apply a fixed coefficient w_2 to x_2 . The linear model therefore cannot use the value of x_1 to change the coefficient on x_2 and cannot solve this problem. *(Right)* In the transformed space represented by the features extracted by a neural network, a linear model can now solve the problem. In our example solution, the two points that must have output 1 have been collapsed into a single point in feature space. In other words, the nonlinear features have mapped both $\mathbf{x} = [1, 0]^\top$ and $\mathbf{x} = [0, 1]^\top$ to a single point in feature space, $\mathbf{h} = [1, 0]^\top$. The linear model can now describe the function as increasing in h_1 and decreasing in h_2 . In this example, the motivation for learning the feature space is only to make the model capacity greater so that it can fit the training set. In more realistic applications, learned representations can also help the model to generalize.

→ In modern Neural networks, ~~and~~ it is recommended to use the rectified linear unit or ReLU, defined by the activation fn.

$$g(z) = \max\{0, z\}$$

i.e $f(x; W, c, w, b) = w^T \max\{0, w^T x + c\} + b$ — (1)

Now will solve XOR Problem

Let $W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

$c = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$, $w = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$ & $b = 0$

→ Let X be i/p's matrix

i.e $X = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$

first

$XW = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}$

Next, add bias vector c

i.e $\begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$

ie $\begin{bmatrix} 0+0 & 0+(-1) \\ 0+1 & 1+(-1) \\ 0+1 & 1+(-1) \\ 0+2 & 2+(-1) \end{bmatrix}$

In this space, all the examples lie along slope 1, As we move along this line, the o/p needs to begin at 0, then rise to 1, then drop back down to 0.

A linear model cannot implement such a fn.

→ To compute 'h' here we need to apply the rectified linear Transformation.

$$\text{i.e. } \max(0, w^T x + c) + b$$

$$f(x; w, c, w, b) = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} \begin{matrix} \xrightarrow{\max(0,0) (0,-1)} = 0, 0 \\ \xrightarrow{\max(0,1) (0,0)} = 1, 0 \\ \vdots \end{matrix}$$

Now these points lie in a space where a linear model can solve the problem

→ Now multiply this by weight vector $w = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$

i.e. $\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$

$$\begin{aligned} 0 \times 1 + 0 \times -2 &= 0 \\ 1 \times 1 + 0 \times -2 &= 1 \\ 1 \times 1 + 0 \times -2 &= 1 \\ 2 \times 1 + 1 \times -2 &= 0 \end{aligned}$$

which (solves) gives the XOR output for every example in the training data.

==